

실시간 마켓데이터 FEED 플랫폼

MDFeed

거래소 실시간 시세를 수집 → 정규화 → 멀티프로토콜 배포하는 금융 데이터 피드 서비스와,
그것을 리눅스에서 운영하기 위한 자동화 스택

5

수집 경로
실연결 검증

1,783

코스피 종목
계층형 피드

26

실측으로 잡은
결함 기록

179

테스트
CI 6잡 통과

목차

Contents

01.

무엇을 만들었나

거래소마다 다른 형식
FEED 서비스의 정의
수집 경로 5개 · 자산군 4축

04.

실측으로 잡은 결함

문서가 틀린 곳
측정이 거짓말하던 곳
돌려봐야 드러난 곳

02.

어떻게 접근했나

막아야 할 실패 4가지
구조와 격리 설계
결과 · 배운 것

05.

어디까지 검증했나

성능 · 부하 곡선
장애 주입 · 장시간 운전
데이터 품질

03.

채용 요건 대응

요건 → 근거 → 실측
직접 구현한 네트워크·IPC
우대사항 2건

06.

판단과 범위

직접 구현 vs 오픈소스
정직하게 적는 한계
연락처

01. 무엇을 만들었나

What.

금융정보 서비스는 거래소 시세를 받아 고객 단말에 공급합니다. 그런데 **거래소마다 형식이 전부 다릅니다**. 이것을 하나로 통일해 여러 소비자에게 나눠주는 중간 계층이 **FEED 서비스**입니다. 매매 프로그램도, 시세 화면도 아닙니다. **그 화면들에 데이터를 먹여주는** 뒷단입니다.

거래소마다 다른 형식

업비트	JSON을 바이너리 프레임으로 보냄
바이낸스	가격을 문자열로 보냄
한국투자	005930^103546^266250^... 파이프·캐럿 구분 문자열

수집 경로 — 5개 전부 실연결 검증

암호화폐	업비트 · 바이낸스 WS	7	실시간 틱
국내주식 (틱)	KIS WS H0STCNT0	3	실시간 체결
국내주식 (전체)	KIS REST 계층형	1,783	15분 회전
지수	KIS 업종 현재지수	6	10초
해외금리	KIS 금리 종합	7	5분

구조



핵심 경로 의존성 0. 피드 계층을 직접 만든 부수 효과로, python3 만 있으면 어떤 리눅스에서도 git clone && make demo 로 바로 돍니다. 설치 문서가 필요 없습니다.

02. 어떻게 접근했나

Approach.

채용 요건의 대부분이 이 계층에서 나옵니다 — 프로세스 간 통신, TCP/HTTP, 리눅스 서비스 점검, 파이프라인 자동화. 요건에 "안다"고 쓰는 대신 **동작하는 것으로 보이기로** 했습니다. 기능을 늘리는 것이 아니라 **네 가지 실패를 막는 것**을 목표로 정의했고, 이 정의가 이후 모든 판단의 기준이 됐습니다.

문제 01 세션은 반드시 끊긴다 끊긴 구간은 영구 유실. 거래소는 과거를 다시 주지 않는다 시퀀스 갭 탐지 + 스냅샷 복구	문제 02 느린 소비자가 전체를 멈춘다 한 구독자를 기다리면 전원이 굼는다 구독자별 유한 큐 · 드롭 카운트	문제 03 프로세스는 멀쩡한데 데이터만 안 온다 실제 장애의 대부분. <code>systemctl status</code> 로 안 잡힌다 liveness / readiness 분리 + 위치독	문제 04 틀린 값을 조용히 배포한다 값이 없는 것보다 나쁘다. 아무 알람도 안 울린다 품질 검사 6종 상시 집행
--	--	--	---

결과 Result	무결성 48.5% → 100.0000% · IPC 지연 p50 47.3 μs · 파이프라인 49,235 msg/s · 적재 480,586 rows/s · 품질검사 108,156건 중 CRITICAL 0 · 장애 주입 4종 복구 확인
-------------------------	---

배운 것 Insight	① 문서를 믿기 전에 한 번 재본다 — KIS 문서는 체결구분 3 이 매도라 했지만 실제로는 5 였습니다. ② 근거 없이 적은 한계 서술은 없느니만 못하다 — "구독자 수백이면 재인코딩이 병목"이라 적어 뒀는데 재보니 CPU 0.28% 였습니다. ③ 계측 도구도 거짓말을 한다 — 알람 11개 중 4개가 영원히 안 울릴 상태였고, 장애 주입 테스트 하나는 아예 안 돌고 통과하고 있었습니다.
----------------------------	--

03. 채용 요건 대응

Skills.

금융 데이터 FEED 개발·운영	수집 경로 5개, 배포 프로토콜 3종, 스냅샷+델타 복구	무결성 48.5% → 100.0000%
Linux 서비스·프로세스 점검·안정화	systemd 6유닛 + 자동 검증 테스트베드 + 장애 주입	11항목 · 복구 4종 확인
파이프라인·배포·점검 자동화	Makefile 단일 진입 · CI 6잡 · Pages 지표 자동 갱신	테스트 179개
Python	수집부터 hot path ·운영 도구까지 전부	8,835줄 · 핵심 의존성 0
SQL 기초 · 관계형 DB 이해	복합 인덱스 · 시간 집계 upsert · 하이퍼테이블 · 보존정책	적재 480,586 rows/s
Linux 명령·프로세스·로그 확인	ops.sh 8종 진단 · RUNBOOK 장애 8종	1차 진단 한 줄
프로세스 간 통신, 네트워크 프로토콜	MDFP/1 TCP · WebSocket 클라이언트+서버 · HTTP/1.1 서버 · UDS pub/sub · 공유메모리 링버퍼 직접 구현	IPC 지연 p50 47.3 μs
자료구조 · 운영체제 · 네트워크	링버퍼 · 로그버킷 히스토그램 · 시그널 · AIMD 유량 제어	세그폴트·좀비 실사례
우대 — 파이프라인·인프라·자동화	SEC EDGAR · OpenDART 수집기 · 16.3일 무중단 운영	487만4천건 · error 0
우대 — Feed 기반 마켓데이터 서비스	KIS 실계좌 연결 · 계층형 피드 설계 · 품질 감시	코스피 1,783종목

직접 구현한 네트워크 · IPC — 각각이 다룬 문제

MDFP/1 TCP

메시지 경계 없는 스트림 · 길이 프리픽스 · CRC32 · 갭 탐지

WebSocket 클라이언트+서버

RFC 6455 · 핸드셰이크 · 마스크 · 단편화 재조립

HTTP/1.1 서버

keep-alive · 바이트 정확한 Content-Length · 경로 탈출 차단

UDS pub/sub 버스

프로세스 간 팬아웃 · 구독자별 백프레서 · 지수 백오프

공유메모리 링버퍼

SPSC 브로드캐스트 · 논블로킹 · 추월 감지 · 찢긴 읽기 방지

04. 실측으로 잡아낸 결함 — 26건 중 대표 5건

Debugging.

이 프로젝트에서 가장 값진 것은 기능 목록이 아닙니다. 문서가 틀린 곳, 측정이 거짓말하던 곳, 실제로 돌려봐야 드러난 곳입니다. 전부 참조 클라이언트 · 통합 테스트 · CI · 장애 주입에서 나왔습니다.

① 구독자가 받은 데이터의 절반이 무결성 실패 — 48.5%

게이트웨이가 심볼을 걸러 보내면서 시퀀스는 전역 번호를 그대로 흘렸습니다. 구독자 입장에선 갈 데 없는 갭이 계속 보입니다. 구독자별로 시퀀스를 다시 매기도록 고쳤습니다. → 100.0000%

② 국내 매도 체결이 전부 방향 미상

KIS 문서는 체결구분 3 이 매도라고 명시합니다. 300건을 호가와 대조하니 5 가 매도였고 3 은 한 건도 없었습니다. 그대로 뒀다면 주문 흐름 지표가 통째로 무의미해졌을 것입니다. → 관측 기준으로 매핑, 미매핑 코드는 세어서 노출

③ 지연시간이 -30,825 μs — 음수로 측정됨

거래소 시계가 로컬보다 앞섭니다. 음수 지연은 어떤 SQL 집계로도 되살릴 수 없습니다. NTP 방식 최소값 필터로 venue별 오프셋을 추정해 보정했습니다. → 정상 범위 복귀

④ 벤치마크가 지연이 아니라 큐 적체를 측정하고 있었다

최대 속도로 밀어 넣고 지연을 재니 p50 105 ms. 지연이 아니라 기다린 시간이었습니다. 페이스 부하와 버스트 처리량으로 분리했습니다. → 지연 p50 47.3 μs

⑤ 알람 11개 중 4개가 영원히 안 울릴 상태였다

카운터를 0으로 초기화하지 않으면 첫 이벤트 전까지 시계열 자체가 없고, Prometheus는 없는 값을 0 이 아니라 no data 로 봅니다. 시작 시점에 카운터를 미리 선언하도록 고쳤습니다. → 알람 검증기를 CI에 편입

이 목록이 이 프로젝트의 알맹이입니다. 기능은 코드를 읽으면 압니다. 하지만 "문서가 틀렸다"를 어떻게 알았는가, "내가 쓴 한계 서술이 틀렸다"를 어떻게 발견했는가는 과정을 남겨야만 전달됩니다.

05. 어디까지 검증했나 — 전부 실행해서 얻은 값

Results.

실측 성능

계층	처리량	p50	p99
MDFP 인코딩	2,305,570 msg/s	434 ns	—
MDFP 파싱 (재동기 포함)	759,786 msg/s	1,316 ns	—
공유메모리 링버퍼	1,560,551 msg/s	641 ns	—
UDS 순수 IPC 지연	—	47.3 μs	105.9 μs
HTTP/1.1 (keep-alive)	15,237 req/s	62.2 μs	95.5 μs
전체 파이프라인	49,235 msg/s	—	—

부하 — 어디서 무너지는가

구독자	총 처리량	1인당 유지율	p99 지연	유실	드롭
1	40 msg/s	100%	1.4 ms	0	0
25	1,330	139%	8.0 ms	0	0
100	6,376	168%	27.9 ms	0	0

처리량은 버팁니다. 총 처리량이 160배로 늘어도 유실·드롭이 0이고, 먼저 무너지는 건 지연입니다(p99 19배). "몇 ms까지 허용되나"를 답할 근거입니다.

이 측정이 제 가설을 뒤집었습니다. 설계 문서에 "구독자 수백 명이면 재인코딩이 병목"이라 적어 뒀는데, 100명에게 6,376건을 뿌려도 재인코딩 CPU는 **0.28%**였습니다. 단계를 쪼개니 구독자당 재인코딩 552 ns, 큐 삽입 410 ns, 소켓 write **1,476 ns — 전체 비용의 59%**였습니다. 근거 없이 적은 한계 서술은 그럴듯해서 아무도 의심하지 않고, 그 위에 잘못된 최적화 계획이 쌓입니다.

05. 장애가 나도 돌아오는가

Reliability.

복구 경로는 평소에 안 돌아갑니다. 그래서 **일부러 고장을 냈습니다**. 61분을 그냥 돌렸을 때 재접속이 0회였다는 건, 복구 코드가 맞는지 확인된 적이 없었다는 뜻입니다.

108,156

품질 검사 건수

0

CRITICAL

0.050%

암묵 환율 자산 간 괴리

0.01

60분 RSS 증가 MB/h (최대)

11

systemd 자동 검증 항목

TCP 스트림에 쓰레기 주입

오검 3회 → **재동기 4회 후 100% 계속 수신**. 손상된 프레임 하나를 버리고 나머지를 살립니다

읽지 않는 구독자 25초

게이트웨이 정상 유지. 느린 하나가 전체를 굶기지 않고, **버린 건수는 지표로 노출**됩니다

feedd SIGKILL

writer PID 유지 → 수신 재개 (38,047 → 39,975). Wants= 로 바뀌 **연쇄 재시작을 없앴습니다**

장시간 연속 운전

60분 · 8개 서비스 전부 **RSS 증가 최대 0.01 MB/h · FD 누수 0**

인프라 실검증

Ubuntu 22.04 테스트베드 systemd 11항목 · Docker Compose 7컨테이너 healthy · Prometheus 8타깃 · 알람 11개 4그룹 · Grafana 12패널

데이터 품질 — 원칙을 집행하는
부분

가격 점프 · 크로스된 호가 · 광폭 스프레드 · 정체 · OHLC 위반 · 교차 시장 괴리 6종 상시 검사. 외부 환율 없이 **업비트 KRW ÷ 바이낸스 USD**로 자체 정합성을
봅니다

06. 판단과 범위

Decision.

직접 만든 것이 많다는 사실 자체는 자랑이 아닙니다. **어디를 직접 만들고 어디를 검증된 도구에 맡길지** 판단한 근거가 이 프로젝트의 내용입니다.

직접 구현 vs 오픈소스

피드 프로토콜 직접 구현	라이브러리를 써도 메시지 프레임িং · 시퀀스 갭 탐지 · 스냅샷 복구 · 느린 구독자 격리는 어차피 직접 해야 합니다. 그 경계를 알려면 한 번은 아래까지 내려가 봐야 합니다.
분석 · 백테스트 오픈소스	<code>backtesting.py</code> · <code>vectorbt</code> 가 낫습니다. 우리가 제공하는 건 데이터와 전략 코드 이고, 검증은 검증된 도구에 맡깁니다.
저장소 오픈소스	PostgreSQL + TimescaleDB. 시계열 파티셔닝 · 보존정책을 직접 만들 이유가 없습니다.
관측 오픈소스	Prometheus + Grafana. 우리가 만든 건 무엇을 지표로 내보낼지 와 알람 규칙입니다.

정직하게 적는 범위

국내 주식 틱은 3종목까지	계좌 한도로 실측해 확인했습니다. 나머지 1,780종목은 REST 스냅샷으로 덮되 체결 단위가 아님 을 명시합니다.
호가는 최우선(BBO)만	전체 depth 20호가 복원은 범위 밖입니다.
단일 노드 단일 리전	다중 리전 이중화, PTP 하드웨어 타임스탬프는 범위 밖입니다.

정직하게 적습니다. 못 하는 것을 한 척하지 않는 것이 이 프로젝트의 원칙입니다. 데이터 서비스에서 **틀린 값은 없는 값보다 나쁘고**, 그 원칙은 문서에도 똑같이 적용됩니다.

검토해주셔서 감사합니다

Thank you.

저장소	github.com/oyeong011/market-feed-platform
데모	oyeong011.github.io/market-feed-platform
참조 데이터	github.com/oyeong011/financial-database
메일	duddls5079@gmail.com